```
  def initialize(rank, suit)
    @rank = rank
    @suit = suit
  end
end

def pick_a_card(cards)
  cards = cards.sort_by(&:rank)
  case cards
  in [{rank:}, {rank: ^(rank + 1)}, {rank: ^(rank+ 2)}] if rank > 6
    "You have three consecutive high cards"
  else
    "You have no interesting cards,"
  end
end

puts pick_a_card([
  Card.new(7, "Hearts"),
  Card.new(8, "Diamonds"),
  Card.new(9, "Clubs")
])
```

produces:

```
You have no interesting cards,
```

The problem is that {rank:} no longer matches anything because Card isn't a hash.

Happily, Ruby provides a way for this to work, the Card class can implement a method called deconstruct_keys that returns a Hash version of the class suitable for pattern matching.

Here we use the hash shortcut to define our deconstruct_keys method:

```
class Card
  attr_accessor :rank, :suit

  def initialize(rank, suit)
    @rank = rank
    @suit = suit
  end

  def deconstruct_keys(keys)
    {rank:, suit:}
  end
end

def pick_a_card(cards)
  cards = cards.sort_by(&:rank)
  case cards
  in [{rank:}, {rank: ^(rank + 1)}, {rank: ^(rank+ 2)}] if rank > 6
    "You have three consecutive high cards"
  else
    "You have no interesting cards,"
  end
end

puts pick_a_card([
  Card.new(7, "Hearts"),
  Card.new(8, "Diamonds"),
  Card.new(9, "Clubs")
```